

Görsel Yönetim ve Toyota Üretim Sistemi: Zemin İşaretlemelerinden Mieruka Kavramına Tematik Bir İnceleme

Ömer Faruk Yavuz

Temmuz 2026

Görünürlük İlkesinin Yazılıma Aktarımı: Toyota Üretim Sistemin- den Mühendislik Pratiğine

Çerçeve

Ana tez ve aktarım sorusu

Yalın üretim literatürünün merkezî tezi şudur: sistemin asıl teknolojisi makine ya da yazılım değil, *görünürlüktür*. Zemine çekilen bir çizgiden *mieruka* () kavramına uzanan bütün araçlar aynı önermenin farklı soyutlama düzeylerindeki ifadeleridir — bir üretim ortamı, ancak normal durumun standartlaştırıldığı ve anormalliğin derhâl fark edilebildiği ölçüde yönetilebilir.

Bu bölümün sorusu, tezin kendisi değil transferidir: *fabrika zemininde geçerli olan bu ilke, kodun üretildiği ortamda hangi biçimde ve hangi kayıplarla geçerlidir?*

Soru retorik değildir, çünkü transfer zaten olmuştur. Kanban panoları iş akışı yönetiminde, WIP limitleri akış kontrolünde, andon ilkesi uyarı sistemlerinde, gemba ilkesi gözlemlenebilirlik pratiğinde, standartlaştırılmış iş ise işletim runbook'larında karşılığını bulmuştur. Ancak transfer seçici olmuştur ve seçimin kendisi çoğunlukla bilinçsizdir: kanban yazılıma *heijunka*'sız geçmiştir — WIP limiti korunmuş, talebi dengeleme disiplini aktarılmamıştır.

Bu bölümün amacı, seçimi bilinçli hâle getirmektir.

Aktarım ölçütü

Bir aracın yazılıma aktarılabilirliği üç testle değerlendirilmektedir.

1. **Kavramsal karşılık testi.** Aracın dayandığı olgunun yazılımda bir karşılığı var mıdır? “Envanter” kavramının karşılığı vardır (birleştirilmemiş kod); “raf ömrü” kavramının karşılığı zayıftır.
2. **Disiplin testi.** Karşılık kurulduğunda, aracı işleten disiplin de birlikte taşınmakta mıdır, yoksa yalnızca biçim mi aktarılmaktadır? Pano kurmak kolaydır; panonun gösterdiği anormalliğe müdahale etme zorunluluğu aktarılmadığında pano dekordur.
3. **Ters etki testi.** Araç, yazılımın kendine özgü koşullarında ters yönde çalışır mı? Kapasite kullanımını yükseltmek fabrikada erdemdir; kuyruk kuramının geçerli olduğu bilgi işinde doğrudan zarardır.

Bu üç teste göre araçlar üç sınıfa ayrılmaktadır: **doğrudan aktarılabilir, uyarlanarak aktarılabilir, aktarılamaz veya yanıltıcı.**

Sınıf	Ölçüt
Doğrudan	Üç testi de geçer; kavram, disiplin ve etki korunur
Uyarlanarak	Kavram geçer, disiplin yeniden kurulmalıdır
Yanıltıcı	Kavram benzer görünür, etki terstir

Birinci Küme: Görünürlük ve Anormallik Tespiti

Mieruka'nın yazılımdaki karşılığı

Görsel yönetimin ölçütü şudur: iyi örgütlenmiş bir işyerinde herhangi bir gözlemci, yerinden çıkmış bir paleti saniyeler içinde tespit edebilmelidir. Yazılımdaki karşılığı doğrudandır ve gözlemlenebilirlik (observability) disiplininin tanımıdır: sistemin iç durumu, dışarıdan üretilen sinyaller üzerinden, önceden tanımlanmamış sorular sorulabilecek biçimde okunabilir olmalıdır.

Zemin işaretlemelerinin renk sözlüğü, yazılımda üç ayrı yüzeyde karşılık bulmaktadır:

- **Depo düzeyi.** Dizin yapısı ve modül sınırları — neyin nereye ait olduğunun sözlü açıklamaya gerek kalmadan okunabilmesi. Yalın literatürdeki *gölge pano* (her parçanın dış çizgisinin çizildiği, eksik parçanın iki saniyede görüldüğü düzen) kavramının karşılığı, standart proje iskeleti ve yapılandırma dosyalarının öngörülebilir konumudur.
- **Çalışma zamanı düzeyi.** Log seviyeleri, servis sağlık rozetleri, hata bütçesi göstergeleri. *Kırmızı/yeşil etiket* pratiğinin doğrudan karşılığı, servisin dağıtımına uygun olup olmadığını tek bakışta bildiren durum göstergesidir.
- **Akış düzeyi.** İş panosu, dağıtım hattı durumu, açık olay listesi.

Üç somut kaizen fikrinin karşılığı

Popüler yalın literatürdeki üç uygulama, yazılımda düşük maliyetli ve yüksek getirili karşılıklara sahiptir.

Günlük üç problem listesi: günün yalnızca en büyük üç probleminin panoya yazılması. Yazılımdaki karşılığı, ekip kanalında sabitlenmiş “bugünün üç engeli” notudur ve backlog’un genişliğine karşı odak üretir.

En çok görülen beş ret nedeni panosu: yazılımda, üretim ortamındaki en sık beş hata türünün panoda tutulması ve günlük güncellenmesi. Hata kuyruğunun toplam sayısı değil, dağılımın tepesi görünür kılınır — Pareto ilkesinin operasyonel biçimi.

Beş satırlık vardiya devri: çıktı, problemler, destek, plan, notlar. Nöbet devri (on-call handover) için doğrudan şablondur ve devir teslimde en sık yaşanan bilgi kaybını — “dün gece ne oldu” sorusunun cevapsız kalmasını — kapatır.

İkinci Küme: Akış, Kuyruk ve Parti Büyüklüğü

Envanterin epistemolojik işlevi

Just-in-Time’in kuramsal önemi, envantere yüklediği epistemolojik işlevdedir: geleneksel yaklaşım stoku bir sigorta olarak kullanır; sistem ise bu sigortanın problemleri *gizlediğini* ileri sürer. Envanter azaltıldıkça akışı bozan yapısal sorunlar görünür olur.

Yazılımda envanterin karşılığı nettir ve aktarımın en güçlü noktasıdır:

Üretimde	Yazılımda
Yarı mamul (WIP)	Birleştirilmemiş dal, açık PR
Bitmiş ürün stoku	Yayımlanmamış, sürümü kesilmemiş özellik
Parti büyüklüğü	PR boyutu, sürüm kapsamı
Temin süresi	Commit'ten üretime geçen süre
Fire / hurda	Terk edilmiş dal, geri alınmış değişiklik
Aşırı üretim	Talep edilmemiş özellik geliştirme

Ohno'nun aşırı üretimi *birincil* israf sayması, yazılımda birebir geçerlidir: kullanılmayan özellik yalnızca geliştirme maliyeti değil, sonrasında bakım, test yüzeyi, dokümantasyon ve bilişsel yük olarak zincirleme maliyet üretir.

Küçük parti ilkesi

Karşılaştırmalı üretim sonuçları belirgindir: aynı günlük talep düzeyinde sürekli akış, parti üretimine kıyasla yarı mamul stokunu ve temin süresini onda birin altına indirebilmektedir. Yazılımdaki karşılığı sürekli entegrasyon ve gövde tabanlı geliştirmedir; küçük ve sık birleştirilen değişiklikler, uzun ömürlü dalların ürettiği birleştirme çatışması ve gecikmiş geri bildirim maliyetini ortadan kaldırır.

Bu, aktarımın en iyi belgelenmiş kısmıdır ve deneysel destek taşır: dağıtım sıklığı ile değişiklik başarısızlık oranı arasındaki ilişki, küçük parti lehinedir.

Aktarılmayan disiplin: heijunka

Kanban yazılıma talebi dengeleme disiplini olmaksızın geçmiştir. Fabrikada heijunka, model karışımının ve hacmin zamana yayılarak düzenlenmesidir; amacı *mura* (dengesizlik) giderilmeden *muri* (aşırı yük) ve *muda* (israf) çözülemeyeceği tespittir.

Yazılımda karşılığı, iş türlerinin karışımını bilinçli olarak dengelemektir: her sprint veya döngü, sabit oranlarda özellik geliştirme, hata giderme, bakım ve altyapı işi içermelidir. Bu disiplin uygulanmadığında panoya WIP limiti konulması dengesizliği çözmez, yalnızca kuyruğu görünür kılar.

Takt süresi: yanıltıcı aktarım

Takt süresi, kullanılabilir zamanın müşteri gereksinimine bölünmesiyle elde edilen ve üretim temposunu talebe kitleleyen ölçüdür. Geçerlilik koşulu, çıktı biriminin türdeş olmasıdır.

Yazılım işi türdeş değildir. Takt kavramının “sprint başına şu kadar puan” biçimindeki aktarımı, ölçüyü hedefe dönüştürerek tahmin enflasyonu ve iş bölme oyunları üretir. Sağlıkta yalın literatüründeki sınır burada da geçerlidir: takt, talebin standartlaştırılamadığı süreçlerde ancak kısmi uygulama bulur.

Aktarılabilir olan takt değil, takt'ın *teşhis işlevi*dir: akış süresinin dağılımına bakıp hangi aşamanın darboğaz olduğunu görmek. Ölçü hedef değil, gösterge olarak kullanılmalıdır.

Üçüncü Küme: Kaynağında Kalite

Jidoka ve dağıtım hattı

Jidoka, anormal durumun derhâl tespit edilip sürecin durdurulması ve anormalliğin ardıl sürece geçirilmemesidir. Muayene ekonomisine dayanan mantığı şudur: hat sonunda örneklemeye yapılan muayene kusursuzluğu garanti edemez ve katma değer üretmez.

Yazılımdaki karşılığı doğrudandır ve sistemin en başarılı aktarımıdır:

Üretimde	Yazılımda
Anormallikte duran makine	Kırmızı build'de duran hat
Andon ipi	Dağıtımı durdurma yetkisi
İlk parça onayı	Kanarya dağıtımı, aşamalı yayım
Geçer/kalmaz mastarı	CI kapısı, eşik kontrolü
Poka-yoke	Tip sistemi, linter, pre-commit kancası
Hata önleyici fişktür	Kod üretici, proje iskeleti
Dört göz ilkesi	İki onaylayan kuralı, eş programlama
Öz kontrol	PR açmadan önce kendi kendine inceleme
Eş kontrolü	Kod incelemesi
Barkod kontrolü	Sağlama toplamı, içerik hash'i, imza doğrulama

Poka-yoke'nin tasarım boyutu özellikle önemlidir: kalite muayenesinin değil tasarımın sonucudur; hedef, hatanın oluşumunu fiziksel olarak olanaksız kılan tasarımdır. Yazılımdaki en güçlü biçimi, geçersiz durumun temsil edilemez kılınmasıdır — tip sistemi aracılığıyla yanlış durumun derleme aşamasında reddedilmesi, çalışma zamanında kontrol edilmesinden yapısal olarak üstündür.

Kalite kapıları ve mimarisi

Kalite yönetiminin üç düzeyi ayrımı yazılıma aynen aktarılır: kalite güvence (QA) süreci iyileştirerek kusuru önler ve proaktiftir; kalite kontrol (QC) üründe kusuru tespit eder ve reaktiftir; kalite yönetim sistemi (QMS) örgüt genelindeki çerçevedir.

Yazılım kültüründe “QA” sıklıkla yalnızca ikinci anlamda — test etme — kullanılmakta, birinci anlam kaybolmaktadır. Ayrımın korunması, “daha çok test yazalım” refleksiyle “hatanın oluşmasını engelleyecek biçimde tasarlayalım” tercihini birbirinden ayırır.

İlk geçiş verimi ve bileşik verim

İki üretim metriği yazılıma beklenenden iyi oturmaktadır.

First Pass Yield (FPY): yeniden işlem görmeden ilk seferde doğru üretilen ürün oranı. Yazılımdaki karşılığı, ilk denemede yeşil geçen build oranı veya ilk incelemede ek düzeltme istenmeden birleşen PR oranıdır.

Rolled Throughput Yield (RTY): bütün aşamalardaki bileşik hatasız geçiş verimi. Dağıtım hattına doğrudan uygulanır ve öğretici bir sonuç üretir: her aşaması yüzde doksan beş başarılı olan altı aşamalı bir hattın uçtan uca ilk geçiş başarısı yaklaşık yüzde yetmiş dördtür. Hattın uzunluğu, aşama başına başarı oranından bağımsız olarak bir maliyet kalemidir.

Dördüncü Küme: Standartlaştırılmış İş

Standartlaştırılmış iş, her görev için bilinen en iyi yöntemin belgelenmesidir; amacı katılık değil, iyileştirme için istikrarlı bir taban oluşturmaktır. Standart olmayan süreçte iyileştirme ölçülemez.

Yazılımdaki karşılıkları katmanlıdır:

- **Kod düzeyi.** Biçimlendirici ve linter yapılandırması, adlandırma sözleşmeleri, kabul edilmiş kalıplar.
- **Süreç düzeyi.** PR şablonu, tanımlanmış bitmiş kriteri, sürüm kontrol listesi.
- **İşletim düzeyi.** Runbook, olay müdahale prosedürü, nöbet devir şablonu.
- **Karar düzeyi.** Mimari karar kaydı (ADR) — kararın kendisini değil, gerekçesini standartlaştırır.

Adler'in “öğrenen bürokrasi” bulgusu bu kümede belirleyicidir: standart, yukarıdan dayatıldığında denetim aracıdır ve yabancılaştırır; standardı yazma ve değiştirme yetkisi işi yapana verildiğinde, aynı standart öğrenmenin altyapısına dönüşür. Belirleyici değişken formalizasyon derecesi değil, formalizasyonun kimin elinde olduğudur.

Yazılıma uygulanmış hâli şudur: linter kuralını mimarlık kurulu koyuyorsa denetimdir; kuralı yazan ve tartışmayla değiştiren, o kodu yazan ekipse öğrenme altyapısıdır. Aynı dosya, farklı yönetimle zıt işlev görür.

Ohno'nun formülasyonu bu ilişkinin dinamik niteliğini tamamlar: bugünün en iyi yöntemleri yarın eskiyecektir; felsefe sabit kalırken yöntemler sürekli iyileştirilir. *Shu-ha-ri* modeli bunun pedagojik kuramıdır — kural önce aynen korunur, sonra bilinçli olarak esnetilir, sonunda aşılır; ve aşma ancak hâkimiyetten sonra meşrudur.

Beşinci Küme: Kök Neden ve Öğrenme

4M çerçevesinin olay analizine uyarlanması

Üretimdeki 4M kontrol listesi (Man, Machine, Material, Method) olay sonrası incelemeler için doğrudan uyarlanabilir bir yapı sunmaktadır:

Boyut	Yazılımda sorulacak sorular
İnsan	Nöbetçi eğitilmiş miydi, yorgun muydu, runbook'a erişebildi mi, iletişim açık mıydı
Makine	Altyapı sağlıklı mıydı, CI koşucuları, izleme sistemi, uyarı kanalı çalışıyor muydu
Malzeme	Girdi verisi, bağımlılık sürümü, yapılandırma, harici servis yanıtı beklenen miydi
Yöntem	Runbook güncel miydi, dağıtım prosedürü izlendi mi, değişiklik yönetimi işletildi mi

Çerçevenin değeri, incelemenin refleks olarak tek boyuta — genellikle “insan hatası” — kilitlenmesini engellemesidir.

İnsan hatasının çözülmesi

Kök neden analizinde “insan hatası” etiketinde durmak metodolojik bir yanılgıdır. Üretim literatürünün üçlü ayrımı yazılımda aynen geçerlidir:

- **Düşünme hataları.** Bilgi temelli (kural ve rutinin bulunmadığı olağandışı durum) ve kural temelli (geçerli kuralın yanlış uygulanması). Karşı önlem: prosedür güncellemesi, araç iyileştirmesi, duruma dönük eğitim.
- **Eylem hataları.** Sık tekrarlanan eylemin aksaması ve kısa süreli bellek kaynaklı unutma. Karşı önlem: kontrol listesi, dikkat dağıtıcıların giderilmesi, bağımsız çapraz denetim — yazılımda üretim ortamına erişimde iki kişi onayı, tehlikeli komutlarda açık doğrulama.
- **Kasıtlı sapmalar.** İstisnai, durumsal ve rutinleşmiş uyumsuzluk. Karşı önlem: kestirme yolu ödüllendiren teşviklerin kaldırılması ve gerçekçi olmayan iş yüküne ilişkin geri bildirim mekanizması.

Üçüncü kategori yazılımda özel bir öneme sahiptir: süreç atlatma davranışı — testi devre dışı bırakma, incelemesiz birleştirme, üretime elle müdahale — neredeyse her zaman *rutinleşmiş* biçimde görülür ve teşvik yapısının semptomudur. Kişiye değil, teslim baskısına bakılmalıdır.

CAPA çevrimi ve olay yönetimi

Üretimdeki CAPA uygulama adımları, olay müdahale sürecinin neredeyse birebir karşılığıdır:

1. Problemi açık biçimde tanımla.
2. Etkilenen bileşeni kontrol altına al — yazılımda: geri alma, bayrak kapatma, trafik yönlendirme.
3. Kullanıcı, sistem ve güvenlik üzerindeki etkiyi değerlendir.
4. Gerçekleri ve süreç verilerini topla — log, iz, metrik, zaman çizelgesi.
5. Kök neden analizi yap.
6. Düzeltici faaliyeti tanımla — mevcut problemi giderir.
7. Önleyici faaliyeti tanımla — tekrarını engeller.
8. Sorumlu kişiyi ve hedef tarihi belirle.
9. Uyula.
10. Etkinliği veriyle doğrula.
11. Runbook, çalışma talimatı ve kontrol planlarını güncelle.
12. Etkilenen ekibi bilgilendir.
13. Kanıtla birlikte kapat.

Yazılım pratiğinde en sık atlanan adımlar altıncı ile yedinci arasındaki ayırım ve onuncu adımdır. Düzeltici ile önleyici faaliyetin ayrılmasa, olayın kapatılmasıyla problemin çözülmesinin karıştırılmasına yol açar. Etkinliğin veriyle doğrulanmaması ise aynı olayın altı ay sonra tekrar açılmasının en yaygın nedenidir.

Hansei ve yokoten: en az aktarılan kavramlar

Popüler yalın aktarımında sistematik bir sapma gözlenmektedir: araç envanterleri hipertrofik, öz-düşünüm kavramları ise fiilen yoktur. Neyin viral olduğunun haritası, neyin öğrenilmediğinin haritasıdır.

Hansei (), derin öz-eleştiri pratiğidir ve ayırt edici özelliği başarı durumunda dahi işletilmesidir; “problem yok” beyanı bu çerçevede problemin kendisi sayılır, çünkü ya gözlemin yetersizliğine ya da standardın gevşekliğine işaret eder. Yazılımdaki karşılığı, yalnızca olay sonrası değil, *sorunsuz geçen* sürümlerin de incelenmesidir — neyin işe yaradığı, hangi kontrolün fiilen devreye girdiği, hangisinin şansla atlatıldığı.

Yokoten (), yatay yayılım, tek bir ekipte geliştirilen çözümün diğerlerine bilinçli transferidir. Yazılımdaki karşılığı, olay incelemelerinin ilgili ekiple sınırlı kalmaması ve çıkan karşı önlemin benzer topolojiye sahip diğer servislere sistematik olarak uygulanmasıdır. Yokoten olmaksızın kök neden analizi, birbirinden habersiz yerel düzeltmeler toplamı olarak kalır.

Altıncı Küme: Malzeme Listesi ve Bağımlılık Yönetimi

Beklenmedik biçimde en yüksek aktarım oranına sahip küme, malzeme listesi (BOM) terminolojisidir. Karşılık tesadüfi değildir: yazılım tedarik zinciri güvenliği alanında kullanılan SBOM (Software Bill of Materials) kavramı doğrudan bu gelenekten devşirilmiştir.

Üretimde	Yazılımda
BOM	Bağımlılık manifestosu, SBOM
Bileşen	Paket
Alt montaj	Kütüphane, iç modül
BOM seviyesi	Geçişli bağımlılık derinliği
BOM açılımı	Bağımlılık ağacı
Malzeme kodu	Paket tanımlayıcısı
Revizyon seviyesi	Sürüm, kilit dosyası
Alternatif malzeme	Yedek uygulama, geri düşüş
Raf ömrü	Destek penceresi, EOL tarihi
Kullanımdan kalkmış malzeme	Terk edilmiş bağımlılık
Kritik malzeme	Tek nokta bağımlılığı
Onaylı tedarikçi listesi	İzinli kayıt defteri, lisans beyaz listesi
Üret veya satın al	Kendi yaz veya kütüphane kullan
Geçerlilik tarihi	Sürüm yayım tarihi
Değişiklik emri (ECO)	RFC, mimari karar kaydı
Spesifikasyon	Arayüz sözleşmesi, şema
Muayene seviyesi	Güvenlik taraması kapsamı
Emniyet stoku	Yerel ayna, önbelleklenmiş artefakt

Uygulamaya dönük sonuç şudur: bağımlılık yönetimi bir konfigürasyon işi değil, tedarik zinciri yönetimidir. Üretim disiplininin sorduğu sorular — bu kalem kritik mi, alternatifi var mı, raf ömrü ne zaman doluyor, tedarikçisi onaylı mı — yazılım bağımlılıkları için de sorulmalıdır ve çoğunlukla sorulmamaktadır.

Yedinci Küme: Duruş, Bakım ve Değişim Süresi

Küçük duruşlar ve kararsız testler

Toplam üretken bakım disiplinindeki *küçük duruş* (chokotei) kategorisi — sık tekrarlanan, tekil olarak önemsiz görünen kesintiler — yazılımdaki en doğrudan karşılığını kararsız testlerde bulmaktadır.

Yapısal benzerlik tamdır. Her iki durumda da kesinti tekil olarak küçüktür, bu nedenle kayda geçirilmez; toplamı büyüktür, bu nedenle ölçülemez; ve en yıkıcı etkisi kapasite kaybı değil *güven kaybıdır*. Kararsız test, hattı durduran sinyali gürültüye çevirir ve jidoka mekanizmasının kendisini iptal eder — kırmızı build'in anlamı kalmadığında andon ipi de anlamsızlaşır.

Uygulama kuralı üretimden aynen alınabilir: küçük duruşlar ayrı bir kategori olarak sayılmalı, kök nedeni izlenmeli ve tekrarlananlar kalıcı çözülene kadar haftalık incelenmelidir.

SMED ve dağıtım süresi

SMED disiplini, değişim faaliyetlerini makinenin durmasını gerektiren *iç* faaliyetler ile makine çalışırken yürütülebilen *dış* faaliyetlere ayırır; kazanç, içselin dışsala dönüştürülmesinden gelir.

Yazılımdaki karşılığı dağıtım sürecidir. İç faaliyet, kesinti gerektiren adımdır — şema göçü, önbellek geçersizleştirme, servis yeniden başlatma. Dış faaliyet, sistem çalışırken yapılabilir — imaj oluşturma, artefakt hazırlama, yapılandırma doğrulama, veri arka doldurma.

SMED mantığının yazılıma en uygun uygulaması, genişlet-daralt (expand–contract) göç kalıbıdır: şema değişikliği geriye uyumlu bir genişletme ile dışsallaştırılır, kesinti gerektiren daraltma ise trafiğin tamamı yeni sözleşmeye geçtikten sonra yapılır. İç faaliyet, dış faaliyete dönüştürülmüştür.

Planlı bakım

Duruş azaltma listesindeki “planlı bakım” ve “makinaları temiz tut” maddeleri, yazılımda düzenli bağımlılık güncellemesi, güvenlik yaması ve teknik borç ödemesi olarak karşılık bulur. Üretimdeki gerekçe aynen geçerlidir: güvenilir ekipman, işletmeyi ya ihtiyat envanteri tutmaya ya da fazladan kapasite yatırımına zorlar; bakım disiplini bütün üst katmanların ön koşuludur.

Sekizinci Küme: Ölçüm ve Patolojileri

Doğrudan aktarılabilen metrikler

Üretim metriği	Yazılım karşılığı
Temin süresi	Commit’ten üretime geçen süre
Ret oranı	Değişiklik başarısızlık oranı
Duruş süresi	Kullanılamazlık süresi, MTTR
İlk geçiş verimi	İlk denemede yeşil build oranı
Değişim süresi	Dağıtım süresi
Hurda oranı	Terk edilen dal, geri alınan değişiklik
Stok devir hızı	Açık PR’ın kapanma hızı
Darboğaz	Akıştaki en uzun bekleme aşaması

Yanıtıcı metrikler

İki üretim metriği yazılıma aktarıldığında ters yönde çalışmaktadır.

Kapasite kullanımı. Fabrikada mevcut kapasitenin ne kadarının kullanıldığı bir verimlilik göstergesidir. Bilgi işinde ise kuyruk kuramı geçerlidir: değişkenliğin yüksek olduğu bir sistemde kullanım oranı yüzde yüze yaklaştıkça bekleme süresi doğrusal değil üstel biçimde artar. Yüzde yüz doldurulmuş bir ekip takvimi, verimlilik göstergesi değil temin süresi patlamasının habercisidir. Boşluk, israf değil akış koşuludur.

Toplam ekipman etkinliği (OEE). Gösterge hedefe dönüştüğünde, satılmayacak ürünün üretilmesi dahi göstergeyi yükseltir; aşırı üretim israfı, ölçüm sisteminde başarı olarak görünür. Yazılımdaki karşılığı, geliştirici çıktısının kod satırı, commit sayısı veya kapatılan bilet adediyle ölçülmesidir. Aynı patoloji, aynı mekanizmayla üretilir.

Görünür ve gerçek verimlilik

Üretim literatürünün en yararlı ayrımı budur ve Goodhart yasasının onlarca yıl önceki habercisidir: satış gözetilmeksizin üretim miktarını artırmak yalnızca sayısal düzeyde kalan *görünür* bir verimliliktir; *gerçek* verimlilik, satılabilir miktarın mümkün olan en az kaynakla üretilmesidir.

Yazılımdaki karşılığı, çıktı (output) ile sonuç (outcome) ayrımıdır. Yerel iyileştirmeler daima toplam sistem etkisiyle birlikte değerlendirilmelidir — bir ekibin hızlanması, entegrasyon noktasındaki kuyruğu büyütüyorsa sistem yavaşlamıştır.

Görünürlük rejiminin bütünlüğü buna bağlıdır: yanlış şeyi görünür kılan pano, körlükten daha tehlikelidir, çünkü yanlışla kanıt statüsü kazandırır.

Dokuzuncu Küme: İnsan, Kültür ve Olgunluk

Andon’un sosyolojisi

Andon mekanizmasının işleyişi teknik değil sosyolojik bir koşula bağlıdır: hattı durdurma *hakkı* ile durdurmanın *güvenliği* aynı şey değildir. Ampirik kontrast keskindir — özgün uygulamada ip

günde binlerce kez çekilirken, sistemi biçimsel olarak kopyalayan fabrikalarda ip mevcut ancak atılır.

Yazılımdaki karşılığı doğrudandır ve sınanabilir. Bir ekipte dağıtım durdurma yetkisi resmen tanımlıysa ancak son altı ayda hiç kullanılmamışsa, iki açıklama vardır: ya hiç anormallik yaşanmamıştır ya da durdurmanın bedeli görünmez biçimde durduran kişiye yüklenmektedir. İkincisi kural olarak doğrudur.

Aynı sosyolojik koşul olay incelemeleri için de geçerlidir: suçlamasız inceleme bir prosedür değil, yaşanmış deneyimle kurulan bir güvence rejimidir. Belge suçlamasız olabilir; kariyer sonuçları suçlamalıysa, belge işlevsizdir.

Olgunluk seviyeleri

Üretim mükemmelliği için tanımlanan beş seviyeli model, mühendislik kariyer merdivenine doğru dan oturmaktadır:

Seviye	Yazılımdaki karşılığı
1 — Uygulayıcı	Sözleşmelere uyar, işi doğru yapar
2 — Problem çözücü	Kök nedene iner, kalıcı düzeltir
3 — Süreç iyileştirici	Araç, hat ve akışı geliştirir
4 — Performans lideri	Ekip çıktısının tutarlılığını kurar
5 — İş etkisi lideri	Sistem ve iş modeli düzeyinde karar verir

Modelin taşıdığı ayrım, kariyer çerçevelerinde sıklıkla bulanıklaşan bir noktayı netleştirir: ikinci seviye problemi çözer, üçüncü seviye problemin *sınıfını* ortadan kaldırır. Aynı hatanın tekrar tekrar giderilmesi ikinci seviyede bir başarıdır, üçüncü seviyede bir başarısızlıktır.

Gemba: gözlem disiplini

Genchi genbutsu ilkesi, yönetsel bilginin kaynağına ilişkin epistemolojik bir tercihtir: iş, rapor edildiği yerde değil gerçekleştiği yerde gözlemlenmelidir. Yöntemsel ilkesi, gözlemin denetime dönüşmemesidir; çalışanlar denetlendiklerini sezdikleri anda davranışları temsile dönüşür.

Yazılımdaki karşılıkları üç düzeydedir: kullanıcının ürünü fiilen kullanımının izlenmesi, geliştiricinin geliştirme ortamıyla kurduğu sürtünmenin gözlenmesi, ve sistemin üretim ortamındaki fiili davranışının — test ortamındaki değil — incelenmesi. Üçüncüsü gözlemlenebilirlik disiplininin kendisidir: sorun, log'un anlattığı yerde değil, sistemin çalıştığı yerde aranmalıdır.

Aktarılamayanlar

Dürüstlük gereği, kaynak malzemenin önemli bir bölümünün yazılımda karşılığı bulunmamaktadır veya zorlama karşılıklar üretmektedir.

- **Girdi, süreç içi ve çıktı muayenesi (IQC, IPQC, OQC).** Fiziksel parti örneklemesine dayanır; yazılımda çıktı türdeş olmadığı için örnekleme mantığı geçersizdir.
- **Ölçüm sistemi analizi (MSA).** Ölçüm cihazının tekrarlanabilirlik ve yeniden üretilebilirlik analizi, fiziksel ölçüme özgüdür. Yazılımda en yakın karşılık, metrik toplama hattının kendisinin doğruluğunun denetlenmesidir — nadiren yapılır ancak kavramsal olarak meşhurdur.
- **PPAP, COC, COA.** Müşteri onay ve sertifikasyon süreçleri düzenlemeye tabi alanlar dışında karşılıksızdır.

- **Proses yeterlilik endeksleri (Cp, Cpk).** Sürekli ve normal dağılan bir çıktı değişkeni varsayar. Yazılımda gecikme dağılımları kuyruklu ve çarpıktır; yüzdelik dilim tabanlı hedefler (SLO) doğru araçtır, yeterlilik endeksleri değildir.
- **Raf ömrü, fire faktörü, geriye doğru sarf.** Fiziksel malzeme akışına özgüdür.
- **Takt süresinin hedef olarak kullanılması.** Yukarıda gerekçelendirilmiştir.

Ayrıca ters yönde bir kayıt gereklidir: teknik borç kavramı, Ohno'nun israf taksonomisine tam oturmaz. Yedi israf türü stok, hareket ve beklemeyle ilişkindir; birikmiş tasarım yükümlülüğü ise bunların hiçbirisi değildir — gelecekteki değişim maliyetini artıran, bugün görünmeyen bir yükümlülüktür. Bu, yazılımın kendi kavramsal katkısıdır ve üretim çerçevesine geri aktarılamaz.

Uygulama Kontrol Listesi

Aşağıdaki liste, yukarıdaki çözümlemenin doğrudan uygulanabilir çekirdeğidir. Sıralama, uygulama maliyeti ile beklenen etki oranına göre.

Görünürlük

1. Dağıtım hattının durumu, ekibin tek bakışta gördüğü bir yüzeyde bulunsun.
2. Üretim ortamındaki en sık beş hata türü panoda tutulsun ve günlük güncellensin.
3. Nöbet devri beş başlıkla yapılsın: çıktı, problemler, destek, plan, notlar.
4. Her servisin sağlık durumu ve dağıtıma uygunluğu ikili bir göstergeyle okunabilsin.
5. Depo yapısı öngörülebilir olsun; eksik veya yanlış yerdeki dosya iki saniyede fark edilsin.

Akış

1. Açık PR sayısına üst sınır konulsun ve sınır aşıldığında yeni iş başlatılsın.
2. PR boyutu küçük tutulsun; büyük değişiklik, birleştirilebilir parçalara bölünsün.
3. Uzun ömürlü dal kullanılsın; entegrasyon günlük düzeyde yapılsın.
4. İş türü karışımı bilinçli dengelensin: özellik, hata, bakım ve altyapı için sabit oran belirlensin.
5. Ekip kapasitesinin tamamı planlanmasın; boşluk bir israf kalemi değil akış koşulu sayılsın.

Kalite

1. Kırmızı build üzerine yeni değişiklik birleştirilmesin.
2. Geçersiz durum, çalışma zamanında kontrol edilmek yerine tip düzeyinde temsil edilemez kılınsın.
3. Her yeni sürüm önce sınırlı trafikle yayımlansın — ilk parça onayı ilkesi.
4. Kararsız testler ayrı bir kategori olarak sayılsın, kök nedeni izlensin ve kalıcı çözümlene kadar haftalık incelensin.
5. Kalite kapıları otomatik olsun; insan iradesine bağlı kapı, baskı altında ilk feda edilendir.

Öğrenme

1. Olay incelemesinde düzeltici ve önleyici faaliyet ayrı ayrı yazılsın.

2. Her karşı önlemin sorumlusu ve hedef tarihi olsun; etkinliği veriyle doğrulanmadan olay kapatılmasın.
3. İnceleme çıktısı benzer topolojiye sahip diğer servislere sistematik olarak taşısın.
4. Yalnızca başarısız değil, sorunsuz geçen sürümler de incelensin.
5. “Problem yok” beyanı, gözlemin yetersizliği ihtimaline karşı sorgulansın.

Tedarik zinciri

1. Bağımlılık envanteri (SBOM) üretilsin ve sürümlensin.
2. Kritik ve tek nokta bağımlılıkları açıkça işaretlensin.
3. Her önemli bağımlılığın destek penceresi ve alternatifi kayıtlı olsun.
4. İzinli kayıt defteri ve lisans politikası tanımlansın.
5. Bağımlılık güncellemesi planlı bakım olarak takvimlensin, kriz anına bırakılmasın.

Yönetişim

1. Standartları yazma ve değiştirme yetkisi, o standartla çalışan ekipte olsun.
2. Dağıtımı durdurma yetkisinin fiilen kullanılıp kullanılmadığı ölçülsün.
3. Bireysel çıktı metrikleri performans değerlendirmesine bağlanmasın.
4. Yerel iyileştirme kararları, toplam akış etkisiyle birlikte değerlendirilsin.
5. Aynı problemin tekrar tekrar giderilmesi, çözüm değil teşhis eksikliği sayılsın.

Kapanış: Aktarımın Kendi Dersi

Yalın araçların sektörler arası dolaşımı, “araç kopyalanır, felsefe kopyalanmaz” tezinin ampirik testini sağlamaktadır. Her transfer, kavram setinin bir alt kümesini taşımakta ve taşınmayan kısım genellikle aracı işleten disiplin olmaktadır.

Yazılıma yapılan aktarımda bu örüntü nettir. Kanban panosu taşınmış, talebi dengeleme disiplini taşınmamıştır. Kalite kapısı taşınmış, kapıyı çalıştıran durdurma yetkisi taşınmamıştır. Kök neden analizi taşınmış, yatay yayılım taşınmamıştır. Ölçüm araçları taşınmış, “görünür verimlilik” uyarısı taşınmamıştır.

Ortak nokta şudur: aktarılan her seferinde *görünür ve sayılabilir* olan yüzey, aktarılmayan ise onu anlamlı kılan düşünme disiplindir. Bu, popüler aktarımın kendi eleştirdiği hatayı yeniden üretmesidir.

Sonuç, üç kayıtlı bağlanabilir. Birincisi, görünürlük etik açıdan nötr bir teknolojidir; aynı rejim, standardın kimin elinde olduğuna bağlı olarak öğrenmenin altyapısı da olabilir, gözetimin mimarisi de — ve yazılımda bu risk, geliştirici üretkenliği ölçümünün yaygınlaşmasıyla soyut olmaktan çıkmıştır. İkincisi, görünürlük yalnızca bakılan yerde işler; mevcut mimarinin içindeki anormallikleri kusursuz açığa çıkarır, ancak mimarinin kendisinin sorgulanması ayrı bir kurumsal yetenektir. Üçüncüsü, eliminasyona izin verilen tamponlar olağanüstü koşullarda opsiyon değeri taşır; israf tanımı riskten bağımsız bir mutlak değil, riske göre ayarlanan bir değişkendir.

Bu üç kaydı birlikte taşıyabilen bir uygulama, araçları kopyalamış değil, ilkeyi edinmiş olur.